NITBFreshers Study Portal

Functional Documentation

NITBFreshers Portal — nitbfreshers.42web.io

Maulana Azad National Institute of Technology, Bhopal

Developed by:	Devansh Soni (3rd Year, ECE)
Maintained by:	Tanay Sharma (2nd Year, CSE) & Shree Pandit (2nd Year, CSE)
Hosting Platform:	InfinityFree (42web.io)
FTP Client:	FileZilla
Document Version:	1.1
Date:	April 2026

This is a student-run, unofficial initiative.
Scholar Numbers are used solely for portal functionality and personalisation.

Contents

1 Introduction **5**
1.1 Scope . . . 5
1.2 System Overview . . . 5
1.3 Entities . . . 5

2 System Architecture **6**
2.1 Technology Stack . . . 6
2.2 Hosting Infrastructure . . . 6
2.3 Database Architecture . . . 6
2.4 Study Material Filesystem . . . 7
2.5 Request Lifecycle . . . 7

3 Authentication & Session Management **9**
3.1 Overview . . . 9
3.2 Files Involved . . . 9
3.3 `index.php` — Primary Login Handler . . . 9
3.3.1 Purpose . . . 9
3.3.2 Working . . . 9
3.3.3 Traffic Redirection Logic . . . 11
3.3.4 Client Fingerprinting . . . 11
3.4 `index_old.php` and `index_2024SchNO.php` — Legacy Variants . . . 11
3.4.1 Purpose . . . 11
3.4.2 Working . . . 12
3.5 `logout.php` — Session Termination . . . 12
3.5.1 Purpose . . . 12
3.5.2 Working . . . 12
3.6 `change_password.php` — Password Update . . . 12
3.6.1 Purpose . . . 12
3.6.2 Working . . . 12
3.7 `db_connection.php` and `db_connection_new.php` . . . 13
3.7.1 Purpose . . . 13
3.7.2 Working . . . 13
3.8 `access_config.json` and `allowed_users.txt` . . . 13
3.8.1 Purpose . . . 13
3.8.2 Working . . . 13
3.9 `bannedusers.txt` — User Banning . . . 14
3.9.1 Purpose . . . 14
3.9.2 Working . . . 14
3.10 Session Variable Reference . . . 14

4 Student Dashboard & Study Material Browser **15**
4.1 Overview . . . 15
4.2 Files Involved . . . 15
4.3 `dashboard.php` — Production Dashboard . . . 15
4.3.1 Purpose . . . 15
4.3.2 Working — PHP Layer . . . 15
4.3.3 Working — HTML & UI Layer . . . 17

4.3.4 Dark Mode . . . 17
4.3.5 Security Restrictions . . . 17
4.4 `dashboard_1.php` — Intermediate Dashboard Version . . . 18
4.4.1 Purpose . . . 18
4.4.2 Working . . . 18
4.5 `dashboard_old.php` — Legacy Dashboard . . . 18
4.5.1 Purpose . . . 18
4.5.2 Key Differences from Current Dashboard . . . 18
4.6 `fetch_subjects.php` — Subject List API . . . 18
4.6.1 Purpose . . . 18
4.6.2 Working . . . 18
4.7 `fetch_resources.php` — Folder & File API . . . 19
4.7.1 Purpose . . . 19
4.7.2 Working . . . 19
4.8 `dashboard.js` and `dashboard_2.js` — JavaScript Browser Controllers . . . 20
4.8.1 Purpose . . . 20
4.8.2 Working — Core Navigation Functions . . . 20
4.9 `resource_viewer.php` — File Viewer . . . 21
4.9.1 Purpose . . . 21
4.9.2 Working — PHP Layer . . . 22
4.9.3 Working — Rendering Strategy . . . 22
4.9.4 Working — UI Layer . . . 22
4.10 `announcements.json` — Announcement Configuration . . . 23
4.10.1 Purpose . . . 23
4.10.2 Structure . . . 23
4.11 Study Material Addition Workflow . . . 23
4.12 Dashboard Version History . . . 24

5 Attendance Tracker 25
5.1 Overview . . . 25
5.2 Files Involved . . . 25
5.3 `attendance_handler.php` — Attendance Backend . . . 25
5.3.1 Purpose . . . 25
5.3.2 Working — Request Validation . . . 26
5.3.3 Working — Timezone Correction . . . 26
5.3.4 Working — Date Validation . . . 26
5.3.5 Working — Database Upsert . . . 27
5.3.6 Working — Statistical Recalculation . . . 27
5.3.7 Working — Attendance Formula . . . 28
5.3.8 Working — Response . . . 28
5.4 Attendance Status Reference . . . 28
5.5 Database Table — `attendance_logs` . . . 29
5.6 Known Limitations and Future Improvements . . . 29

6 File Upload & Student Contribution System 29
6.1 Overview . . . 29
6.2 Files Involved . . . 30
6.3 `upload_handler.php` — Student Upload Backend . . . 30
6.3.1 Purpose . . . 30

6.3.2 Working . . . 30
6.4 Admin Approval Workflow (via `portal_manager.php`) . . . 31
6.5 Database Table — `uploads` . . . 32

7 Feedback System **33**
7.1 Files Involved . . . 33
7.2 `feedback_config.json` — Form Schema . . . 33
7.2.1 Purpose . . . 33
7.2.2 Structure . . . 33
7.3 `feedback_data.txt` — Response Storage . . . 34
7.3.1 Purpose . . . 34
7.3.2 Format . . . 34
7.4 `feedback_submissions.json` — Deduplication Tracker . . . 34
7.4.1 Purpose . . . 34
7.4.2 Working . . . 34
7.5 Admin — Clearing Feedback Data . . . 34

8 Notice & Announcement System **36**
8.1 Files Involved . . . 36
8.2 `notice_config.json` — Global Popup Notice . . . 36
8.2.1 Purpose . . . 36
8.2.2 Structure . . . 36
8.3 `notice_views.json` — View Tracking . . . 36
8.3.1 Purpose . . . 36
8.3.2 Working . . . 37
8.4 `announcements.json` — Dashboard Announcement Bar . . . 37
8.4.1 Purpose . . . 37
8.4.2 Structure . . . 37
8.4.3 Admin Management . . . 37

9 Admin Panel — Portal Manager **38**
9.1 Overview . . . 38
9.2 Files Involved . . . 38
9.3 Authentication . . . 38
9.4 Configuration Initialisation . . . 39
9.5 Flash Message System . . . 39
9.6 Administrative Sections & Operations . . . 39
9.6.1 Emergency Kill Switch (Maintenance Mode) . . . 39
9.6.2 Global Popup Notice . . . 40
9.6.3 Statistics Dashboard . . . 40
9.6.4 Password Reset Requests . . . 40
9.6.5 Student Contributions (Upload Queue) . . . 41
9.6.6 Feedback System Management . . . 41
9.6.7 Traffic Redirection . . . 41
9.6.8 Announcements Management . . . 42
9.6.9 File Manager . . . 42
9.6.10 User Access Control . . . 42
9.6.11 Student Messages . . . 43
9.7 `maintenance_config.json` — System State . . . 43

10 Password Recovery & Account Assistance **44**
10.1 Overview . . . 44
10.2 `forget_password.php` — Account Assistance Page . . . 44
10.2.1 Purpose . . . 44
10.2.2 Working — Maintenance Check . . . 44
10.2.3 Working — Three Scenarios . . . 44
10.2.4 Working — UI Layer . . . 45
10.3 Database Table — `password_reset_requests` . . . 45

11 Public Landing Page **46**
11.1 `home.php` — Portal Landing Page . . . 46
11.1.1 Purpose . . . 46
11.1.2 Working . . . 46
11.1.3 Design Notes . . . 46

12 Logging & Monitoring **47**
12.1 Logged Events . . . 47
12.2 Database Table — `log_entries` . . . 47
12.3 Client Fingerprinting . . . 47

13 Security Architecture **49**
13.1 Implemented Security Measures . . . 49
13.2 Known Vulnerabilities & Gaps . . . 49

14 Known Issues & Future Roadmap **50**
14.1 Known Issues . . . 50
14.2 Future Roadmap . . . 51

15 Configuration File Reference **52**

16 Conclusion **53**

1 Introduction

This document describes the functional design of the **NITBFreshers Study Portal**, an independently developed web-based platform created to supplement academic resources for students of Maulana Azad National Institute of Technology (MANIT), Bhopal.

The portal was built in response to the absence of a centralised, student-friendly resource hub. It provides authenticated access to previous year question papers (PYQs), lecture notes, subject-wise study material, and an attendance tracking system — all accessible via a Scholar Number-based login.

1.1 Scope

This document covers the functional behaviour of every module of the portal — including the authentication system, study material browser, attendance tracker, admin controls, and supporting infrastructure. It is intended to serve as a reference for future contributors, maintainers, and the developers.

1.2 System Overview

Portal URL: https://nitbfreshers.42web.io
Backup Portal: https://nitbfreshers2.42web.io
Backend Language: PHP (procedural)
Database: MySQL via InfinityFree
Frontend: Tailwind CSS, Lucide Icons, Vanilla JavaScript
Study Material Storage: Server filesystem (`/study_material/`) managed via FileZilla FTP
Target Users: B.Tech students of MANIT Bhopal (Batches 2024–25 onwards)

The portal supports two student groups — **MT** (sections A–E) and **ST** (sections F–J) — with shared infrastructure and configurable traffic redirection to handle server load across multiple hosted instances.

1.3 Entities

The following entities operate within the portal:

- **Student** — Any registered MANIT student with a valid Scholar Number.
- **Portal Admin** — The developer/maintainer with access to server files, the database, and the admin panel.
- **Anonymous Visitor** — Unauthenticated users redirected to the login page.

2 System Architecture

2.1 Technology Stack

Layer	Technology
Backend	PHP (procedural, no framework)
Database	MySQL — hosted on InfinityFree shared servers
Frontend	Tailwind CSS, Lucide Icons, Vanilla JavaScript
Session Management	PHP native sessions + secure HttpOnly cookies
File Storage	Server filesystem under `/study_material/`
FTP Client	FileZilla (used by admin to upload study material)
Web Hosting	InfinityFree (42web.io) — free shared hosting
PDF Viewer	PDF.js (desktop) / Google Docs Viewer (mobile)

Table 1: Technology Stack

2.2 Hosting Infrastructure

The portal runs on **InfinityFree**, a free shared hosting provider. Due to traffic limitations inherent to free hosting, the system is architected across **three hosted instances**:

- **Primary:** `nitbfreshers.42web.io` — main portal for all users.
- **Mirror 1:** `nitbfreshers2.42web.io` — overflow portal for MT group (Sections A–E).
- **Mirror 2:** `nitbfreshers3.42web.io` — overflow portal for ST group (Sections F–J).

Each instance maintains its own copy of the codebase and connects to its own MySQL database. The primary portal contains a **traffic redirection system** (see Section 9) that can route users to mirror portals based on their Roll Number group when server load requires it.

2.3 Database Architecture

Two separate MySQL databases are maintained across primary and mirror instances:

- **Primary DB:** `if0_40290568_student_portal` on `sql100.infinityfree.com`
- **Mirror DB:** `if0_38652527_student_portal` on `sql301.infinityfree.com`

All database connections are made via the `db_connect()` function defined in `db_connection.php`. Below is the simplified Entity-Relationship (ER) model representing the core tables:

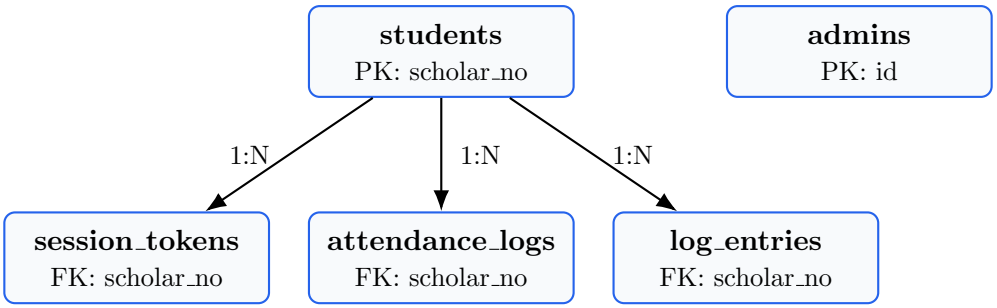

Figure 1: Database schema and core table relationships

Table	Purpose
`students`	Stores scholar number, name, roll number, semester, password
`session_tokens`	Persistent login tokens for "Remember Me" functionality
`log_entries`	Login, logout, and resource access events with IP and OS
`attendance_logs`	Per-subject daily attendance records per student
`uploads`	Pending and approved student file upload records
`admins`	Admin credentials (bcrypt-hashed passwords)

Table 2: Database Tables Description

2.4 Study Material Filesystem

Study material is organised on the server filesystem under the `/study_material/` directory. The admin uploads files using **FileZilla FTP client** directly to the InfinityFree server. The directory structure follows a strict hierarchy:

```
study_material/
MT/
   Mathematics/
     Notes/
       file.pdf
     PYQs/
   Physics/
ST/
    ...
```

The portal dynamically reads this directory structure via `fetch_subjects.php` and `fetch_resources.php` — no database entries are required when new material is added. Adding a folder or file via FileZilla immediately makes it available on the portal.

2.5 Request Lifecycle

A typical student resource access flow proceeds as illustrated in the flowchart below:

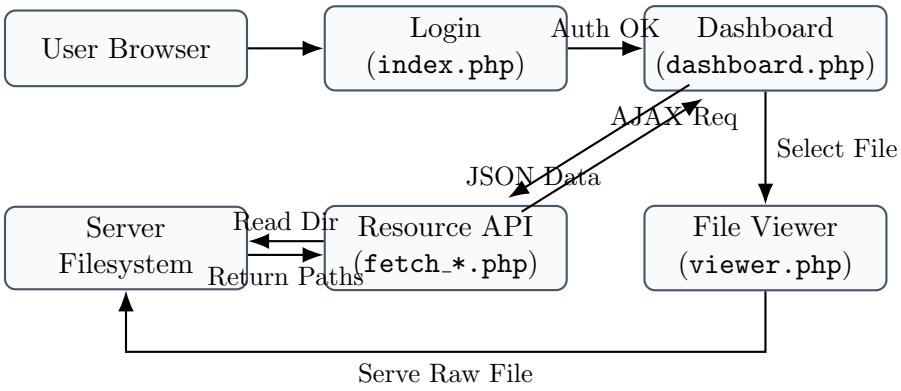

Figure 2: System Request Lifecycle for Study Material Access

1. Student visits the portal and authenticates via `index.php`.
2. PHP session and a secure cookie token are created and stored in `session_tokens`.
3. Student selects Group → Subject → Folder via the dashboard.
4. The browser calls `fetch_subjects.php` and `fetch_resources.php` which scan the filesystem and return JSON.
5. Student selects a file; browser redirects to `resource_viewer.php`.
6. The viewer detects file type and device, selects the appropriate rendering method (PDF.js / Google Docs Viewer / iframe), and logs the access event to `log_entries`.

3 Authentication & Session Management

This module governs all aspects of student identity verification, persistent login, session lifecycle, and password management. It is the entry point for every user interaction with the portal and acts as the security boundary between public and authenticated content.

3.1 Overview

Authentication is implemented entirely in PHP using a dual-layer approach:

- **Layer 1 — PHP Session:** A standard server-side session (`$_SESSION`) is created upon successful login. It holds the student's Scholar Number, name, roll number, and semester for the duration of the browser session.
- **Layer 2 — Persistent Cookie Token:** A cryptographically secure 64-character token is generated, stored in the `session_tokens` database table, and set as a secure HttpOnly cookie (`portal_auth`) with a 7-day expiry. This enables the "Keep me logged in" functionality.

Every protected page begins with a session guard:

```
if (!isset($_SESSION['scholarNo'])) {
   header('Location:  index.php');
   exit;
}
```

Any unauthenticated request to a protected page is immediately redirected to `index.php`.

3.2 Files Involved

3.3 `index.php` — Primary Login Handler

3.3.1 Purpose

This is the entry point of the portal. It serves the login form to unauthenticated visitors and processes login POST requests. It also handles automatic login via the persistent cookie, and triggers group-based traffic redirection after successful authentication.

3.3.2 Working

The file executes in three sequential phases on every page load:

Phase 1 — Cookie Check (Auto Login):

1. The file checks for the existence of the `portal_auth` cookie.
2. If found, the token value is looked up in the `session_tokens` table via a JOIN with the `students` table to also retrieve name, roll number, and semester in a single query.
3. If the token is valid, session variables are populated and the user is redirected to `dashboard.php` without needing to re-enter credentials.

File	Role
`index.php`	Primary login page and authentication handler
`index_old.php`	Legacy login page (older UI, same logic)
`index_2024SchNO.php`	Login variant for 2024-batch Scholar Numbers
`logout.php`	Session destruction and cookie invalidation
`change_password.php`	Authenticated password change form
`forget_password.php`	Password recovery for locked-out students
`db_connection.php`	Primary database connection (used by all auth files)
`db_connection_new.php`	Mirror database connection for secondary server
`redirection_config.json`	Admin-controlled group-based traffic redirection rules
`bannedusers.txt`	Flat-file list of banned Scholar Numbers and reasons
`allowed_users.txt`	Flat-file whitelist (used in restricted access modes)
`access_config.json`	Portal-wide access mode switch (OPEN / RESTRICTED)

Table 3: Authentication Module — Files

4. Before redirecting, the `check_and_perform_redirection()` function is called with the student's roll number to apply group-based routing if the admin has enabled it.

Phase 2 — Form Submission (Manual Login):

1. On POST, both the Scholar Number and password are retrieved and trimmed.
2. **Scholar Number Validation:** The input is validated against three regular expressions before any database query is made:
   - `/^2511[1-9]01[0-1][0-5][0-9]{2}$/` — Standard 2025-batch B.Tech Scholar Numbers.
   - `/^25404011[0-9]{3}$/` — Lateral entry / special batch numbers.
   - `/^254160111[0-9]{2}$/` — MCA / specific programme numbers.

   If the Scholar Number does not match any pattern, login is rejected immediately with *"Invalid Scholar Number format"* without touching the database.
3. A prepared statement queries the `students` table for the given Scholar Number.

4. The submitted password is compared directly against the stored plain-text password field. If matched:
   - `session_regenerate_id(true)` is called to prevent session fixation attacks.
   - A 64-character token is generated via `bin2hex(random_bytes(32))`.
   - The token is inserted into `session_tokens` with a 7-day expiry timestamp.
   - The token is set as a secure, HttpOnly cookie (`portal_auth`).
   - The login event is recorded in `log_entries` with the student's Scholar Number, IP address, and detected OS.
   - Session variables are set: `scholarNo`, `name`, `rollNo`, `semester`, `from_index`.
   - `check_and_perform_redirection()` is called, then the user is redirected to `dashboard.php`.

Phase 3 — Login Form Rendering: If neither cookie nor POST triggers a redirect, the HTML login form is rendered. Error messages from Phase 2 are displayed inline. If the error is an invalid credential or unknown Scholar Number, a *"Forgot Password?"* link to `forget_password.php` is shown.

3.3.3 Traffic Redirection Logic

The function `check_and_perform_redirection($rollNo)` reads `redirection_config.json`. If the admin has set `status` to `"ON"`, the third character of the roll number (0-indexed position 2) is inspected:

- Characters **A–E** → MT group → redirected to `url_ae`.
- Characters **F–J** → ST group → redirected to `url_fj`.

When `status` is `"OFF"` (default), all users proceed to the primary portal regardless of group.

3.3.4 Client Fingerprinting

On every login, the following client metadata is recorded in `log_entries`:

- **IP Address:** Resolved via `HTTP_CLIENT_IP` → `HTTP_X_FORWARDED_FOR` → `REMOTE_ADDR` in priority order to handle proxy setups.
- **Operating System:** Detected by matching the `HTTP_USER_AGENT` string against a regex array covering Windows, macOS, Linux, Android, iOS, and other mobile platforms.

3.4 `index_old.php` and `index_2024SchNO.php` — Legacy Variants

3.4.1 Purpose

These are older versions of the login page maintained for historical compatibility and traffic management across server instances.

3.4.2 Working

- `index_old.php` uses the same Scholar Number regex patterns as the current `index.php` but uses an older UI (Poppins font, minimal styling) and lacks the traffic redirection logic and the OS detection improvements.
- `index_2024SchNO.php` validates Scholar Numbers matching the 2024-batch pattern (`/^2411[1-9]01[0-1][0-5][0-9]{2}$/`) and was the login page used during the portal's first academic year. It does not include redirection logic or the updated OS fingerprinting. The cookie expiry in this version is set to 30 days instead of 7.

Both files share the same `db_connection.php` and redirect to `dashboard.php` on success. They are not linked from the current UI and exist as server-side fallbacks.

3.5 `logout.php` — Session Termination

3.5.1 Purpose

Cleanly terminates the authenticated session, invalidates the persistent cookie token in the database, and redirects the user to the login page.

3.5.2 Working

1. Checks for an active `$_SESSION['scholarNo']`.
2. Collects the client IP and detects the OS via the same User-Agent regex logic as the login handler.
3. Inserts a `'Logout'` event into `log_entries`.
4. If the `portal_auth` cookie exists, its token is `DELETE`d from the `session_tokens` table, and the cookie is expired immediately by setting its expiry to `time() - 3600`.
5. `session_unset()` and `session_destroy()` are called to fully clear server-side session data.
6. The user is redirected to `index.php`.

3.6 `change_password.php` — Password Update

3.6.1 Purpose

Allows an authenticated student to update their portal password. This is especially important because the default password for all new students is their Scholar Number, making this the primary post-registration security step.

3.6.2 Working

1. The page is session-guarded. Unauthenticated requests are redirected to `index.php`.
2. On GET, the form is rendered with the Scholar Number pre-filled as a read-only field.
3. On POST:

- New password and confirm password fields are compared. A mismatch is rejected immediately without a DB call.
- The current (old) password is fetched from the `students` table using a prepared statement.
- The submitted old password is compared directly (plain-text comparison) against the stored value.
- If matched, an `UPDATE` prepared statement sets the new password for that Scholar Number.
- Success or failure is communicated via an inline message with appropriate colour coding (green / red).

> **Known Limitation:** Passwords are stored and compared as plain text. There is no hashing (e.g. bcrypt) applied to student passwords. This is a recognised security gap and a recommended area for improvement in future versions. The admin account, by contrast, uses bcrypt hashing via `password_hash()` as seen in `reset_admin_used.php`.

3.7 `db_connection.php` and `db_connection_new.php`

3.7.1 Purpose

These files encapsulate the MySQL connection logic into a single reusable `db_connect()` function. Every PHP file in the portal includes one of these files and calls `db_connect()` to obtain a `mysqli` connection object.

3.7.2 Working

- `db_connection.php` connects to the **primary** database on `sql100.infinityfree.com`.
- `db_connection_new.php` connects to the **mirror** database on `sql301.infinityfree.com`.
- If the connection fails, `die()` is called with the error message, halting execution immediately.
- The connection object is returned to the caller, which is responsible for closing it with `$conn->close()` after use.

3.8 `access_config.json` and `allowed_users.txt`

3.8.1 Purpose

These two files together implement a **portal-wide access control switch** that allows the admin to restrict portal access to a whitelist of Scholar Numbers during testing, maintenance windows, or beta rollouts.

3.8.2 Working

- `access_config.json` holds a single key: `"mode"` with value `"OPEN"` or `"RESTRICTED"`.
- When `"OPEN"`, all valid Scholar Numbers can log in normally.

- When `"RESTRICTED"`, the login handler cross-references the submitted Scholar Number against `allowed_users.txt`. Only Scholar Numbers listed in that file are permitted to proceed.
- `allowed_users.txt` stores entries as comma-separated pairs: `ScholarNo,Label` (e.g. `25118011289,Devansh Ji ki Jai`).

3.9 `bannedusers.txt` — User Banning

3.9.1 Purpose

Provides the admin with a lightweight, file-based mechanism to restrict specific students from accessing certain portal features without removing them from the database.

3.9.2 Working

- The file stores one entry per line in the format: `ScholarNo,Reason`.
- On dashboard load, the file is read line-by-line and the logged-in student's Scholar Number is compared against each entry.
- If a match is found, `$isBanned = true` and the ban reason is stored in `$bannedReason`.
- Banned students can still log in and view study material, but face specific restrictions based on admin configuration.
- On first login after being banned, a JavaScript `alert()` displays the ban reason to the student (triggered by the `from_index` session flag).
- The file is currently empty, meaning no students are banned.

3.10 Session Variable Reference

The following session variables are set on successful login and consumed across all protected pages:

Variable	Example Value	Usage
`$_SESSION['scholarNo']`	`25116011109`	Primary user identifier across all pages
`$_SESSION['name']`	`Aryan Singh`	Display name on dashboard
`$_SESSION['rollNo']`	`250116109`	Used for group detection and redirection
`$_SESSION['semester']`	`4`	Displayed on student details panel
`$_SESSION['from_index']`	`true`	Triggers ban alert on first dashboard load

Table 4: Session Variables

4 Student Dashboard & Study Material Browser

The Student Dashboard is the core functional interface of the portal. It serves as the central hub from which students access all portal features — study material, announcements, student details, group selection, and navigation to sub-modules. The Study Material Browser, embedded within the dashboard, implements a three-tier dynamic navigation system (Group → Subject → Folder → File) backed entirely by the server filesystem, with no hardcoded content anywhere in the codebase.

4.1 Overview

Upon successful authentication, the student is redirected to `dashboard.php`. This file performs three responsibilities simultaneously:

1. **Session validation and data hydration** — Verifies the session is active and fetches any missing student details from the database.
2. **Access control enforcement** — Checks whether the student is banned and handles the ban notification flow.
3. **UI rendering** — Renders the full dashboard HTML including the student details panel, announcements bar, group selector, and the dynamic study material browser panels.

The study material browser itself is driven by two PHP API endpoints (`fetch_subjects.php` and `fetch_resources.php`) and one JavaScript controller file (`dashboard.js` or `dashboard_2.js` depending on version), which communicate asynchronously via `fetch()` API calls and dynamically manipulate the DOM.

4.2 Files Involved

4.3 `dashboard.php` — Production Dashboard

4.3.1 Purpose

This is the primary interface students interact with after login. It is a single-page PHP-rendered HTML document that combines student information display, announcements, and the full study material browser into one cohesive view. It uses Tailwind CSS for styling and Lucide Icons for iconography, matching the visual language of all other modern pages on the portal (`index.php`, `change_password.php`, `resource_viewer.php`).

4.3.2 Working — PHP Layer

Step 1 — Session Guard: The file immediately calls `session_start()` and checks for `$_SESSION['scholarNo']`. If absent, the user is redirected to `index.php` and execution halts.

Step 2 — Session Data Hydration: Session variables `name`, `rollNo`, and `semester` are checked for the fallback value `'Loading...'`. This fallback is set during login when the JOIN query in `index.php` fails to retrieve these fields (e.g. due to a transient DB error). If any of these values are `'Loading...'`, a fresh SELECT query is executed against the `students` table using the Scholar Number, and the session variables are updated with the

File	Role
`dashboard.php`	Current production dashboard — Tailwind CSS UI
`dashboard_1.php`	Intermediate version dashboard — MT-default group selection
`dashboard_old.php`	Legacy dashboard — older CSS-based UI, full feature parity
`dashboard.js`	Primary JavaScript controller for the study material browser
`dashboard_2.js`	Alternate JS controller (used by `dashboard_1.php`)
`fetch_subjects.php`	API endpoint — returns subject list for a given group
`fetch_resources.php`	API endpoint — returns folders and files for a given path
`resource_viewer.php`	Full-page file viewer for PDFs, PPTs, and URL resources
`announcements.json`	Admin-managed list of announcements shown on dashboard
`style.css`	Legacy stylesheet used by `dashboard_old.php` and `dashboard_1.php`

Table 5: Dashboard & Study Material Browser — Files

retrieved values. This self-healing mechanism ensures the dashboard never displays stale or missing data.

Step 3 — Ban Check: The file reads `bannedusers.txt` line by line. Each line is split on the first comma into Scholar Number and reason. If the logged-in Scholar Number matches any entry:

- `$isBanned` is set to `true`.
- `$bannedReason` is populated.
- The loop breaks immediately.

The `$showAlert` flag is derived from the `from_index` session variable. If both `$isBanned` and `$showAlert` are true, a JavaScript `alert()` is injected into the rendered HTML to notify the student of their restriction and its reason. The `from_index` flag is then unset so the alert does not repeat on subsequent page loads.

Step 4 — Announcements: The file reads `announcements.json` and decodes it into a PHP array. Each announcement object contains a `text` field and a `link` field. These are passed to the frontend where JavaScript renders them as clickable links inside the announcements bar. The announcements bar also contains a close button that hides the

entire bar from the DOM on click, using `this.parentNode.style.display='none'`.

4.3.3 Working — HTML & UI Layer

The dashboard UI is structured into the following regions:

Navigation Bar (Sticky): A sticky top navigation bar contains the portal title, a dark/light mode toggle button, a link to `change_password.php`, and a logout link to `logout.php`. On mobile, the navigation collapses gracefully using Tailwind responsive classes.

Student Details Panel: Displays the authenticated student's name, Scholar Number, roll number, and semester — all retrieved from session variables and escaped via `htmlspecialchars()` before output to prevent XSS.

Announcements Bar: A horizontally scrollable or block-level bar rendered from `announcements.json`. Includes static links to PYQ banks and any dynamically loaded announcement entries. A close () button allows the student to dismiss it for the session.

Group Selector: A `<select>` dropdown with options `MT` and `ST`. In `dashboard_1.php`, MT is pre-selected by default. Changing the selection fires the `updateSubjects()` JavaScript function.

Three-Panel Browser System: The study material browser is built around three mutually exclusive `<div>` panels that are shown and hidden via JavaScript:

- **Panel 1 — Subject Panel (`#subjectPanel`):** Shows subject tiles fetched from the server for the selected group. Each tile is a clickable card with the subject name.
- **Panel 2 — Resource Selection Panel (`#resourceSelectionPanel`):** Shows folder tiles for the selected subject. A Back button returns to Panel 1.
- **Panel 3 — Folder Display Panel (`#folderDisplayPanel`):** Shows individual files and sub-folders within the selected folder. A Back button returns to Panel 2.

4.3.4 Dark Mode

Dark mode is implemented using Tailwind's `darkMode: 'class'` configuration. On page load, the JavaScript checks `localStorage.getItem('theme')`. If `'dark'` or if the system preference is dark and no explicit preference is stored, the `dark` class is added to `document.documentElement`. The toggle button switches between modes and persists the preference to `localStorage`.

4.3.5 Security Restrictions

The dashboard injects a `blockRestrictedActions()` function that:

- Disables the browser right-click context menu via `contextmenu` event interception.
- Blocks F12 (DevTools), Ctrl+S (Save Page), Ctrl+Shift+I (Inspector), and Ctrl+U (View Source) keyboard shortcuts.
- Prevents drag-and-drop of page elements.

> **Note:** These restrictions are client-side only and can be bypassed by a technically proficient user. They serve as a deterrent against casual content copying rather than a hard security boundary. The actual content protection relies on server-side session guards and the file serving architecture.

4.4 `dashboard_1.php` — Intermediate Dashboard Version

4.4.1 Purpose

An intermediate iteration of the dashboard introduced to serve the MT group specifically. It retains the older `style.css`-based UI but adds MT as the default pre-selected group.

4.4.2 Working

Functionally identical to `dashboard_old.php` with one key difference:

1. The group `<select>` element has `MT` as the default `selected` option, and a `window.onload` handler calls `updateSubjects()` immediately to auto-populate subjects without requiring user interaction.

4.5 `dashboard_old.php` — Legacy Dashboard

4.5.1 Purpose

The original dashboard used during the portal's first semester. It has identical PHP-layer logic to the current dashboard (session hydration, ban check, announcements) but uses the `style.css` stylesheet instead of Tailwind CSS. It remains on the server as a rollback option and is not linked from the current UI.

4.5.2 Key Differences from Current Dashboard

- Uses `style.css` with Material Icons Sharp instead of Tailwind CSS and Lucide.
- The group selector defaults to `-- Select --` rather than auto-loading MT subjects.
- Includes an experimental DevTools detection loop (commented out) that attempts to close the browser window if window dimensions suggest DevTools are open.
- Announcement links are hardcoded inline rather than loaded from `announcements.json`.

4.6 `fetch_subjects.php` — Subject List API

4.6.1 Purpose

A lightweight PHP API endpoint that accepts a `group` query parameter and returns a JSON array of subject names available for that group. It serves as the bridge between the filesystem and the JavaScript browser UI.

4.6.2 Working

1. Receives `$_GET['group']` (e.g. `MT` or `ST`).

2. Sanitises the input using `basename()` to prevent directory traversal attacks. This ensures a value like `../../etc` cannot escape the `study_material/` directory.
3. Constructs the directory path: `study_material/{group}`.
4. Verifies the path is a valid directory using `is_dir()`.
5. Uses `glob($directory . '/*')` filtered through `array_filter(..., 'is_dir')` to retrieve only subdirectories (subjects) within the group folder.
6. Maps the full paths to bare folder names using `basename()`.
7. Returns the result as a JSON-encoded array. If the directory doesn't exist or no group is provided, an empty array `[]` is returned.

> **Important Design Decision:** This endpoint does not require authentication (`session_start()` is called but no session guard is applied). The subject names themselves are not sensitive, but this is worth noting. The actual file content is protected by `resource_viewer.php` which enforces the session guard strictly.

4.7 `fetch_resources.php` — Folder & File API

4.7.1 Purpose

A more complex API endpoint that handles two different query modes — fetching folders within a subject, and fetching files and sub-folders within a specific folder. It additionally logs every resource browsing event to the database with IP address and OS fingerprinting.

4.7.2 Working

The file constructs file paths from three GET parameters: `group`, `subject`, and `folder` (optional). All values are URL-decoded via `urldecode()` before use.

Mode 1 — No Subject, No Folder (Fetch All Subjects): Returns all subdirectory names under `study_material/{group}/`. This mode is a fallback and is not actively used by the current UI (which uses `fetch_subjects.php` for this purpose).

Mode 2 — Subject Provided, No Folder (Fetch Folders):

1. Constructs the path `study_material/{group}/{subject}/`.
2. Verifies it is a valid directory.
3. Uses `glob()` filtered to directories only to retrieve all folder names.
4. Returns a JSON object: `{"folders": [...]}`.
5. Logs a `'View Folders'` event to `log_entries` if the session Scholar Number is available.

Mode 3 — Subject and Folder Provided (Fetch Files & Sub-folders):

1. Constructs the path `study_material/{group}/{subject}/{folder}/`.
2. Calls the helper function `getFolderContent($folderPath)` which:
   - Uses `glob($folderPath . '/*')` filtered with `is_file()` to collect files.

- Uses the same filtered with `is_dir()` to collect sub-folders.
- Returns both as a structured array: `{"files": [...], "folders": [...]}`.

3. Returns the full content object as JSON.
4. Logs a `'View Folder Content'` event to `log_entries`.

Client Fingerprinting in Fetch: Unlike most API endpoints, `fetch_resources.php` includes the full IP detection and OS detection logic (identical to `index.php` and `logout.php`) and calls `log_event()` on every folder or file listing request. This gives the admin a detailed trail of exactly which subjects and folders each student browsed, timestamped and tied to their IP and device.

4.8 `dashboard.js` and `dashboard_2.js` — JavaScript Browser Controllers

4.8.1 Purpose

These files implement the entire client-side logic of the study material browser. They manage the three-panel navigation system, communicate with the PHP API endpoints, dynamically build the DOM, handle state persistence across page navigations, and manage the dark mode toggle.

4.8.2 Working — Core Navigation Functions

`updateSubjects()`:

1. Reads the current value of the group `<select>` element.
2. Clears the `#subjectsContainer` div.
3. Calls `fetch(‘fetch_subjects.php?group=$group‘)` asynchronously.
4. On success, iterates the returned array and for each subject:
   - Creates a `div.subject-tile` element.
   - Creates an `<img>` element with `src=./images/{subject.toLowerCase()}.png` for a subject-specific icon.
   - Creates an `<h3>` with the subject name.
   - Attaches an `onclick` handler that calls `showResourceSelection(subject)`.
   - Appends the tile to the container.
5. On failure or empty response, displays an appropriate error message in the container.

`showResourceSelection(subject)`:

1. Hides Panel 1 (`#subjectPanel`) and the subjects section.
2. Shows Panel 2 (`#resourceSelectionPanel`).
3. Updates the panel heading to reflect the selected subject name.
4. Stores the subject name as a `data-subject` attribute on the panel element for later retrieval.

5. Calls `fetchFolders(subject)`.

`fetchFolders(subject)`:

1. Clears `#resourceDisplayContainer`.
2. Calls `fetch('fetch_resources.php?group=$group&subject=$subject')`.
3. For each returned folder, creates a `div.subject-tile` with the folder name and an `onclick` that calls `updateResourceContent(folder)`.

`updateResourceContent(selectedFolder)`:

1. Hides Panel 2, shows Panel 3 (`#folderDisplayPanel`).
2. Retrieves the stored subject from the panel's `data-subject` attribute.
3. Constructs the full API URL with group, subject, and folder all URL-encoded.
4. Calls `fetch(url)` and on success:
   - Clears `#folderContent`.
   - For each file in `data.files`: Creates a `button.file-button` with `onclick` calling `openFile(file, subject, group, selectedFolder)`.
   - For each folder in `data.folders`: Creates a `button.folder-button` with `onclick` calling `openFolder(folder)`.

`openFile(fileName, subject, group, folder)`:

1. Saves the current navigation state (`group`, `subject`, `folder`) to `sessionStorage` so that the browser back button can restore context.
2. Constructs a URL to `resource_viewer.php` with all four parameters URL-encoded.
3. Redirects the browser to the resource viewer via `window.location.href`.

`restorePreviousState()`: Called on `DOMContentLoaded`. Reads `group`, `subject`, and `folder` from `sessionStorage`. If all three are present (i.e. the user navigated back from `resource_viewer.php`), it re-executes the navigation chain — `updateSubjects()` → `showResourceSelection()` → `fetchFolders()` → `updateResourceContent()` — using nested `setTimeout()` calls to account for async fetch delays. This restores the exact panel state the student was in before opening a file.

Back Navigation:

- `goBackToSubjectPanel()`: Hides Panels 2 and 3, shows Panel 1 and the subjects section.
- `goBackToResourceSelectionPanel()`: Hides Panel 3, shows Panel 2.

4.9 `resource_viewer.php` — File Viewer

4.9.1 Purpose

A full-page authenticated viewer that renders study material files directly in the browser. It supports PDFs, PowerPoint files (`.ppt`/`.pptx`), URL shortcut files (`.url`), and generic

file types. It adapts its rendering strategy based on both the file extension and the client device type (mobile vs. desktop), and logs every file access event to the database.

4.9.2 Working — PHP Layer

Step 1 — Session Guard: Enforces authentication. Unauthenticated requests are redirected to `index.php`.

Step 2 — Parameter Validation: Requires all four GET parameters: `file`, `subject`, `group`, and `folder`. Each is URL-decoded and `+` characters are replaced with spaces to handle URL encoding inconsistencies from different browsers.

Step 3 — Path Construction and Security:

1. The base directory is resolved using `realpath(__DIR__ . "/study_material")` and backslashes are normalised to forward slashes for cross-platform compatibility.
2. The full file path is constructed as: `baseDir/group/subject/folder/file`.
3. `file_exists()` is called on the constructed path. If the file does not exist, execution halts with an error message. This prevents directory traversal — even if a malicious path is crafted, it will not resolve to a real file outside `/study_material/`.
4. The file extension is extracted using `pathinfo($filePath, PATHINFO_EXTENSION)` and lowercased.
5. A `'Access'` event is logged to `log_entries` with the path `group/subject/file` as the details field.

Step 4 — URL File Handling: If the file extension is `.url`, the file is read line by line and the `URL=` prefix line is extracted. If the URL is a Dropbox link, `convertDropboxLink()` replaces `dl=0` with `raw=1` to convert it from a preview link to a direct download/embed link.

4.9.3 Working — Rendering Strategy

The viewer constructs the file's public URL as: `https://{HOST}/study_material/{group}/{subject}/`

It then selects a rendering method based on file type and device:

Mobile detection is performed server-side by matching the `HTTP_USER_AGENT` against `/Mobile|Android|iPhone|iPad/i`.

4.9.4 Working — UI Layer

The viewer renders a navigation bar identical in design to all other Tailwind pages. It displays the file path breadcrumb (`group / subject / filename`) in the centre of the navbar. On mobile, the breadcrumb and controls stack vertically. The viewer iframe occupies the full remaining viewport height below the navbar, calculated via CSS: `calc(100vh - 100px)` on desktop and `calc(100vh - 140px)` on mobile.

File Type	Device	Rendering Method
`.pdf`	Desktop	PDF.js viewer (`pdfjs/web/viewer.html?file=...`)
`.pdf`	Mobile	Google Docs Viewer iframe (`drive.google.com/viewerng/viewer?url=...`)
`.ppt/.pptx`	Any	Google Docs Viewer iframe
`.url`	Mobile	Direct link button (iframe bypassed)
`.url`	Desktop	iframe with resolved external URL
Other	Any	Direct iframe with the file URL

Table 6: Resource Viewer — Rendering Strategy

4.10 `announcements.json` — Announcement Configuration

4.10.1 Purpose

A JSON flat file that the admin edits directly to manage the announcements displayed on the dashboard. No UI is required to update announcements — the admin modifies this file via FileZilla and the change is live immediately.

4.10.2 Structure

Each entry in the JSON array is an object with two fields:

- `"text"`: The display label for the announcement link.
- `"link"`: The URL the announcement points to.

Example:

```
[{"text": "Access PYQs 2024", "link": "https://nitbfreshers2.42web.io/pyqs2024/"}]
```

4.11 Study Material Addition Workflow

Adding new study material to the portal requires no code changes and no database updates. The complete workflow is:

1. Admin opens **FileZilla** and connects to the InfinityFree FTP server using the account credentials.
2. Navigates to the `/htdocs/study_material/` directory on the remote server.
3. Navigates into the appropriate group folder (`MT/` or `ST/`).
4. Navigates into (or creates) the subject subfolder (e.g. `Mathematics/`).
5. Navigates into (or creates) the resource type subfolder (e.g. `Notes/` or `PYQs/`).
6. Drags and drops the files (PDFs, PPTs, etc.) from the local machine into the remote directory.

7. FileZilla uploads the files via FTP. Once complete, the files are immediately accessible on the portal — `fetch_resources.php` reads the filesystem in real time on every request.

> **No deployment step is required.** The portal has no build process, no cache to invalidate, and no database entries to insert. The filesystem IS the content database.

4.12 Dashboard Version History

File	UI Framework	Key Characteristic
`dashboard_old.php`	Custom CSS (`style.css`)	Original version, manual group select
`dashboard_1.php`	Custom CSS (`style.css`)	MT auto-selected
`dashboard.php`	Tailwind CSS	Current production, dark mode, Lucide icons, modern UI

Table 7: Dashboard Version History

5 Attendance Tracker

The Attendance Tracker is a student-facing utility module that allows students to self-record their daily class attendance on a per-subject basis. It maintains a persistent log in the database and provides real-time calculated statistics — current attendance percentage, number of classes that can be missed while staying above 75%, or number of consecutive classes that must be attended to recover back to 75%. The entire module is designed around the 75% minimum attendance requirement enforced by MANIT Bhopal.

5.1 Overview

The module operates as a client-server system. The dashboard renders the attendance UI (subject selector, date picker, status buttons), and all write and calculation operations are delegated to the `attendance_handler.php` backend via asynchronous POST requests. The backend performs timezone correction, date validation, database upsert, and statistical recalculation before returning a JSON response that the frontend uses to update the UI without a page reload.

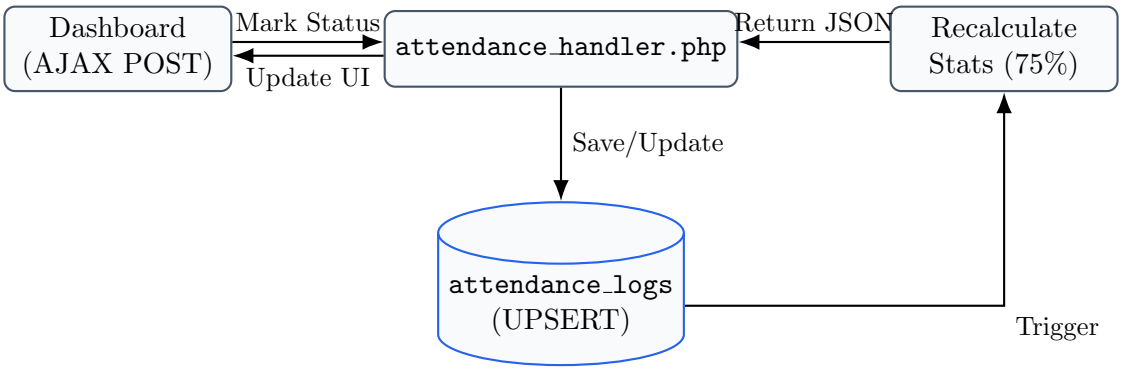

Figure 3: Attendance Tracker - Operation Flow

5.2 Files Involved

File	Role
`attendance_handler.php`	Backend API — handles all attendance mark and recalculate operations
`dashboard.php`	Renders the attendance UI section and initiates AJAX calls

Table 8: Attendance Tracker — Files

5.3 `attendance_handler.php` — Attendance Backend

5.3.1 Purpose

This is the sole backend handler for all attendance operations. It accepts POST requests from the dashboard, validates the input, applies timezone correction, enforces date range

restrictions, performs an upsert operation on the `attendance_logs` table, and recalculates attendance statistics for the affected subject. It returns a structured JSON response for every request.

5.3.2 Working — Request Validation

Step 1 — Session Guard: The handler calls `session_start()` and immediately checks for `$_SESSION['scholarNo']`. If absent, it returns:

```
{"success": false, "message": "Unauthorized"}
```

and halts execution. This ensures the endpoint cannot be called by unauthenticated users even via direct HTTP requests.

Step 2 — Request Method and Action Check: The handler only processes requests where `$_SERVER['REQUEST_METHOD'] === 'POST'` and `$_POST['action'] === 'mark_attendance'`. Any other request is silently ignored.

Step 3 — Input Collection: The following POST fields are read:

- `subject` — The subject name for which attendance is being marked.
- `status` — One of `'Present'`, `'Absent'`, or `'Cancelled'`.
- `date` — Optional. The date for which attendance is being marked (defaults to today if not provided).

Empty `subject` or `status` values are rejected with `{"success": false, "message": "Invalid input"}`.

5.3.3 Working — Timezone Correction

InfinityFree's shared hosting servers operate on a timezone that does not match Indian Standard Time (IST, UTC+5:30). Through observation, the developer determined that the server clock is offset from IST by approximately **13 hours and 30 minutes**. To correct this:

1. A `DateTime` object is created representing the current server time.
2. A `DateInterval` of `PT13H30M` (13 hours, 30 minutes) is added to it.
3. The adjusted date is formatted as `Y-m-d` and used as the effective "today" for all validation logic.

This corrected date is used both as the default value when no `date` POST field is provided, and as the upper bound in the future-date validation check.

> **Design Note:** This is a pragmatic workaround for a hosting limitation. The offset value of +13h30m was empirically derived and hardcoded. If the portal is migrated to a different server with correct IST timezone support, this correction block should be removed.

5.3.4 Working — Date Validation

Two date boundary checks are enforced:

- **Future Date Check:** If `$logDate > $adjustedToday`, the request is rejected with `"Cannot mark future dates"`. This prevents students from pre-marking attendance.
- **Minimum Date Check:** If `$logDate < '2025-12-22'`, the request is rejected with `"Cannot modify before 22 Dec 2025"`. This hardcoded minimum date represents the start of the academic session tracked by the portal, preventing retroactive manipulation of records beyond the system's intended scope.

5.3.5 Working — Database Upsert

Once validation passes, the handler performs an **INSERT ... ON DUPLICATE KEY UPDATE** operation on the `attendance_logs` table:

```
INSERT INTO attendance_logs
(scholar_no, log_date, subject_name, status)
VALUES (?, ?, ?, ?)
ON DUPLICATE KEY UPDATE status = VALUES(status)
```

The table has a composite unique key on `(scholar_no, log_date, subject_name)`. This means:

- If the student has not yet marked attendance for that subject on that date, a new row is inserted.
- If they have already marked it (e.g. they marked Present but want to correct it to Absent), the existing row's `status` is updated in place.

This upsert design makes the operation fully idempotent and eliminates the need for a separate "update attendance" endpoint.

5.3.6 Working — Statistical Recalculation

After a successful upsert, the handler immediately recalculates the student's attendance statistics for that subject across **all recorded dates** (not just the current date). Two separate COUNT queries are executed:

Query 1 — Present Count:

```
SELECT COUNT(*) FROM attendance_logs
WHERE scholar_no = ?  AND subject_name = ?  AND status = 'Present'
```

Query 2 — Total Count (excluding Cancelled):

```
SELECT COUNT(*) FROM attendance_logs
WHERE scholar_no = ?  AND subject_name = ?
AND status IN ('Present', 'Absent')
```

The `'Cancelled'` status is intentionally excluded from the total count. A cancelled class does not count for or against the student — it simply does not exist in the denominator. This correctly models real-world attendance policy where cancelled classes are not penalised.

5.3.7 Working — Attendance Formula

The attendance percentage is calculated as:

$$Percentage = \left(\frac{Present}{Total}\right) \times 100$$

rounded to one decimal place using `round(..., 1)`.

Two advisory messages are then computed based on the 75% threshold:

Case 1 — Below 75%: The number of consecutive classes the student must attend to reach exactly 75% is calculated using:

$$x = \left\lceil \frac{0.75 \times T - P}{1 - 0.75} \right\rceil = \left\lceil \frac{0.75T - P}{0.25} \right\rceil$$

where P is the present count and T is the total count. The message returned is: *"Attend next x classes"*.

Case 2 — At or Above 75%: The number of classes the student can miss while staying at or above 75% is:

$$x = \left\lfloor \frac{P - 0.75 \times T}{0.75} \right\rfloor$$

The message returned is: *"Can miss x classes"*.

5.3.8 Working — Response

On success, the handler returns:

```
{
   "success":  true,
   "percent":  82.5,
   "message":  "Can miss 3 classes"
}
```

The frontend JavaScript on the dashboard receives this JSON response, updates the displayed percentage and advisory message in real time, and applies colour coding (green for ≥75%, red for <75%) without triggering a page reload.

5.4 Attendance Status Reference

Status Value	Meaning	Effect on Calculation
`Present`	Student attended the class	Increments both Present and Total counts
`Absent`	Student missed the class	Increments Total count only
`Cancelled`	Class was cancelled	Excluded from both Present and Total counts

Table 9: Attendance Status Values

Column	Type	Description
`scholar_no`	VARCHAR	FK to `students.scholar_no`
`log_date`	DATE	Date of the class (`Y-m-d` format)
`subject_name`	VARCHAR	Subject name as stored in filesystem
`status`	VARCHAR	One of: `Present`, `Absent`, `Cancelled`

Table 10: `attendance_logs` Table Structure

5.5 Database Table — `attendance_logs`

A composite unique key on `(scholar_no, log_date, subject_name)` enforces the one-record-per-student-per-subject-per-day constraint and enables the upsert pattern.

5.6 Known Limitations and Future Improvements

- **Self-reported data:** Attendance is entirely student-entered with no verification against official records. It serves as a personal tracking tool, not an official record.
- **Hardcoded minimum date:** The `2025-12-22` lower bound will need to be updated or made configurable for future academic sessions.
- **Hardcoded timezone offset:** The +13h30m correction should be replaced with a proper `date_default_timezone_set('Asia/Kolkata')` call if the hosting environment permits it.
- **No timetable integration:** The system does not know which subjects have classes on which days. Students must manually mark each subject independently.

6 File Upload & Student Contribution System

The Student Contribution System allows authenticated students to submit study material files (PDFs, documents, images) for admin review. Submitted files enter a pending queue and are only made available on the portal after the admin explicitly approves and places them into the correct filesystem location. This creates a community-contributed content pipeline with editorial control.

6.1 Overview

The system is split across two roles:

- **Student side:** A form on the dashboard allows file selection, subject tagging, and submission via `upload_handler.php`.
- **Admin side:** The Portal Manager (`portal_manager.php`) presents a pending queue where the admin can preview, approve (choosing the exact target location), or reject each submission.

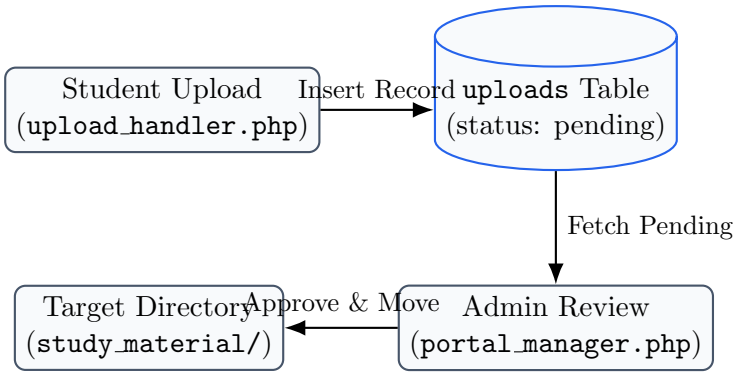

Figure 4: File Upload and Admin Approval Workflow

6.2 Files Involved

File	Role
`upload_handler.php`	Backend API — validates, stores, and records student file uploads
`portal_manager.php`	Admin UI — approval/rejection queue with target path selection
`uploads/pending/`	Temporary server directory for unreviewed uploaded files
`uploads/id_cards/`	Storage for ID card images submitted with password reset requests

Table 11: File Upload System — Files

6.3 `upload_handler.php` — Student Upload Backend

6.3.1 Purpose

Handles all student file upload POST requests. It enforces authentication, validates file type and size, applies a daily upload rate limit, stores the file in a temporary pending directory, and records the submission in the `uploads` database table with status `'pending'`.

6.3.2 Working

Step 1 — Session Guard: Returns `{"status": "error", "message": "Unauthorized"}` and halts if `$_SESSION['scholarNo']` is absent.

Step 2 — Request Method Check: Only POST requests are processed. Any other method returns `"Invalid request"`.

Step 3 — Input Validation: Three POST fields are required: `file` (the file upload), `subject`, and `group`. An optional `folder` field specifies a sub-folder target. Missing fields return `"Missing fields"`.

Step 4 — File Size Validation: Maximum allowed file size is **10MB** (`10 * 1024 * 1024` bytes). Exceeding this returns `"File too large. Max 10MB allowed."`.

Step 5 — File Type Validation: The file extension is extracted using `pathinfo()` and lowercased. Only the following extensions are accepted:

```
pdf, doc, docx, jpg, jpeg, png, ppt, pptx, txt
```

Any other extension returns `"Invalid file type. Only PDF, Docs, and Images allowed."` This prevents executable or script files from being submitted.

Step 6 — Daily Rate Limit: A COUNT query checks how many files the student has uploaded today (matching `scholar_no` and `DATE(upload_date) = today`). If 10 or more uploads are found, the request is rejected with `"Daily upload limit (10 files) reached."`

Step 7 — File Storage: The file is stored in `uploads/pending/` with a unique name constructed as: `time()_uniqid().{ext}`. This prevents filename collisions and obscures the original filename on disk. The directory is created with `mkdir($uploadDir, 0777, true)` if it does not exist.

Step 8 — Database Record: A row is inserted into the `uploads` table via a prepared statement:

```
INSERT INTO uploads
(scholar_no, uploader_name, file_name, stored_name,
subject, group_name, folder_name, file_size, status)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, 'pending')
```

The `file_name` field stores the original filename for display, while `stored_name` stores the unique server-side name. If the DB insert fails, the pending file is immediately deleted with `unlink($destination)` to prevent orphaned files.

Step 9 — Response: Returns `{"status": "success", "message": "File uploaded successfully! Sent for approval."}` on success.

6.4 Admin Approval Workflow (via `portal_manager.php`)

The Portal Manager's Student Contributions section queries all rows in `uploads` where `status = 'pending'` and displays them in a table. For each pending upload, the admin sees: submission date, student name and Scholar Number, original filename (as a clickable link to preview the file), and the originally suggested group/subject/folder fields.

Approve Flow:

1. Admin optionally edits the target group, subject, and sub-folder fields.
2. Admin clicks the Approve button (green checkmark).
3. The handler fetches the upload record from the DB, sanitises the target path components (stripping non-alphanumeric characters), and constructs the target directory: `study_material/{group}/{subject}/[{folder}/]`.
4. The directory is created if it does not exist.
5. If a file with the same name already exists, a timestamp suffix is appended to prevent overwriting.
6. `rename()` moves the file from `uploads/pending/{stored_name}` to the target directory using the original filename.

7. The `uploads` table row is updated to `status = 'approved'`.
8. The file is now immediately accessible to students through the portal's filesystem-based browser.

Reject Flow:

1. Admin clicks the Reject button (red X).
2. The handler fetches the stored filename from the DB.
3. The file is deleted from `uploads/pending/` using `unlink()`.
4. The `uploads` table row is updated to `status = 'rejected'`.

6.5 Database Table — `uploads`

Column	Type	Description
`id`	INT (AUTO)	Primary key
`scholar_no`	VARCHAR	Submitting student's Scholar Number
`uploader_name`	VARCHAR	Student's display name at time of upload
`file_name`	VARCHAR	Original filename as submitted by student
`stored_name`	VARCHAR	Unique server-side filename (`time_uniqid.ext`)
`subject`	VARCHAR	Student-suggested subject
`group_name`	VARCHAR	Student-suggested group (MT/ST)
`folder_name`	VARCHAR	Student-suggested sub-folder
`file_size`	INT	File size in bytes
`status`	ENUM	`pending`, `approved`, or `rejected`
`upload_date`	DATETIME	Server timestamp of submission

Table 12: `uploads` Table Structure

7 Feedback System

The Feedback System is an admin-configurable survey tool that allows the admin to collect structured responses from students. The form schema, question types, and activation state are all managed from the Portal Manager without any code changes. Responses are persisted to a flat file and a JSON tracking file simultaneously.

7.1 Files Involved

File	Role
`feedback_config.json`	Schema definition and on/off status for the feedback form
`feedback_data.txt`	Append-only flat file storing all submitted responses
`feedback_submissions.json`	JSON array of Scholar Numbers who have submitted (deduplication)

Table 13: Feedback System — Files

7.2 `feedback_config.json` — Form Schema

7.2.1 Purpose

This JSON file is the single source of truth for the feedback form. It controls whether the form is shown to students and defines every question, its type, and its options. The admin edits this via the Portal Manager's Form Builder UI.

7.2.2 Structure

```
{
   "status":  "ON" | "OFF",
   "schema":  [
      {
         "id":  "694f93e1bb064",
         "text":  "How can we improve?",
         "type":  "text" | "mcq",
         "options":  "5, 4, 3, 2, 1"
      }
   ]
}
```

Each question object contains:

- `id`: A unique identifier generated via PHP's `uniqid()` at creation time. Used to track responses per question.
- `text`: The question text displayed to the student.
- `type`: Either `"text"` (free-form answer) or `"mcq"` (multiple choice).

- `options`: Comma-separated option values, used only when `type` is `"mcq"`. Ignored for text questions.

When `status` is `"OFF"`, the feedback form is hidden from the dashboard and students cannot submit. The Portal Manager renders a Form Builder interface where the admin can add/remove/reorder questions, toggle types, add MCQ options, and save.

7.3 `feedback_data.txt` — Response Storage

7.3.1 Purpose

An append-only flat file that stores every feedback submission as a single pipe-delimited line. It is human-readable and requires no database query to review.

7.3.2 Format

Each line follows the pattern:

```
TIMESTAMP | ScholarNo | StudentName | Q: QuestionText -> Answer | Q: ...
```

Example from the portal:

```
2025-12-27 09:47:31 | 25116011109 | Aryan Singh Parihar | Q: How can we
improve?  -> Good | Q: How is Shree Pandit Sir?  -> 5
```

The Portal Manager parses each line by splitting on `' | '` with a limit of 4 parts, then displays the four components — timestamp, Scholar Number, name, and response data — in a tabular view. The responses are displayed in reverse chronological order (most recent first) by using `array_reverse()` after reading the file.

7.4 `feedback_submissions.json` — Deduplication Tracker

7.4.1 Purpose

A JSON array of Scholar Numbers that have already submitted feedback for the current form. This prevents a student from submitting multiple responses to the same form instance.

7.4.2 Working

When a student submits feedback:

1. The backend checks if the student's Scholar Number is already in the array.
2. If present, the submission is rejected.
3. If absent, the Scholar Number is appended to the array, and the file is overwritten with the updated array.

The current tracked submissions are: `["25116011109", "25118011289", "25116011101"]` — matching the three entries in `feedback_data.txt`.

7.5 Admin — Clearing Feedback Data

The Portal Manager includes a "Clear All Data" button in the Feedback section. On POST with the `clear_feedback_data` action:

1. `feedback_data.txt` is overwritten with an empty string.
2. `feedback_submissions.json` is overwritten with an empty JSON array `[]`.

This resets both the response log and the deduplication tracker simultaneously, allowing a fresh feedback round to begin.

8 Notice & Announcement System

The portal provides two distinct channels for communicating with students: a **Global Popup Notice** (a modal shown to students once) and a **Dashboard Announcements Bar** (persistent links displayed at the top of the dashboard). Both are managed entirely through flat files — no code deployment is required.

8.1 Files Involved

File	Role
`notice_config.json`	Global popup notice content, status, and unique ID
`notice_views.json`	Per-student record of which notice ID each student has seen
`announcements.json`	List of announcement links displayed in the dashboard bar

Table 14: Notice & Announcement System — Files

8.2 `notice_config.json` — Global Popup Notice

8.2.1 Purpose

Defines a modal popup notice that is shown to students when they load the dashboard. The notice appears only once per student per unique notice content — if the admin changes the title or message, a new unique ID is generated and all students see the notice again.

8.2.2 Structure

```
{
   "status":  "ON" | "OFF",
   "title":  "Result are out !!!",
   "message":  "Visit ERP Portal to see the Report Card",
   "id":  "6952443a60441"
}
```

The `id` field is a `uniqid()` value. It acts as the version identifier for the notice. When the admin updates the notice, the Portal Manager compares the new title and message against the current values. If either has changed, a new `uniqid()` is generated. If neither has changed (e.g. only toggling ON/OFF), the existing ID is preserved — students who have already dismissed the notice will not see it again.

8.3 `notice_views.json` — View Tracking

8.3.1 Purpose

A JSON object mapping each student's Scholar Number to the ID of the last notice they dismissed. This enables the portal to determine whether a student has already seen the current notice.

8.3.2 Working

1. On dashboard load, the portal reads `notice_config.json` to check if `status` is `"ON"`.
2. If ON, it reads `notice_views.json` and checks if the logged-in Scholar Number maps to the current notice ID.
3. If the Scholar Number is absent or maps to a different (older) ID, the popup modal is displayed.
4. When the student dismisses the modal, an AJAX call (or page reload with POST) writes the student's Scholar Number mapped to the current notice ID into `notice_views.json`.

The current `notice_views.json` contains over 200 Scholar Number entries, all mapping to ID `"6952443a60441"`, confirming that 200+ students have seen and dismissed the "Result are out" notice. This file grows unboundedly and may require periodic pruning as new academic years begin.

8.4 `announcements.json` — Dashboard Announcement Bar

8.4.1 Purpose

Defines the list of clickable announcement links shown in the horizontal announcements bar at the top of the dashboard. Unlike the popup notice, announcements are always visible (until the student closes the bar for their session) and can include multiple entries simultaneously.

8.4.2 Structure

A JSON array of objects, each with `"text"` (display label) and `"link"` (URL):

```
[
   {"text":  "Access CYQs 2025", "link":  "https://nitbfreshers2.42web.io/pyqs2025/"
   {"text":  "Access PYQs 2024", "link":  "https://nitbfreshers2.42web.io/pyqs2024/"
]
```

8.4.3 Admin Management

The Portal Manager provides an Announcements section with:

- **Add Announcement:** A two-field form (text + optional URL). Submitting appends the new object to the array and overwrites `announcements.json`.
- **Delete Announcement:** Each listed announcement has a delete button. Clicking it sends the array index to the handler, which uses `array_splice()` to remove that element and overwrites the file.

9 Admin Panel — Portal Manager

The Portal Manager (`portal_manager.php`) is the central administration interface for the entire portal. It is a single self-contained PHP file that implements its own authentication system (separate from student login), and provides a comprehensive web-based dashboard for managing every configurable aspect of the portal. It is accessible only to the admin and is not linked from any student-facing page.

9.1 Overview

The Portal Manager consolidates 20 distinct administrative operations across 10 functional sections into a single authenticated interface. It uses a dark-themed Tailwind CSS sidebar layout with section-based navigation using URL fragment anchors, flash messages for operation feedback, and real-time badge counters for pending items.

9.2 Files Involved

File	Role
`portal_manager.php`	Complete admin dashboard — authentication + all management UIs
`reset_admin_used.php`	One-time utility to create/reset the admin bcrypt password
`maintenance_config.json`	Maintenance mode switch and reason message
`redirection_config.json`	JSON-based group redirection config (ST/MT URL routing)
`redirection_status.txt`	Legacy plain-text redirection flag (`OFF`)
`access_config.json`	Portal-wide access mode: `OPEN` or `STRICT`
`bannedusers.txt`	Flat-file banned user list
`allowed_users.txt`	Flat-file whitelist for STRICT mode
`announcements.json`	Announcement data (managed here)
`feedback_config.json`	Feedback form schema (managed here)
`notice_config.json`	Global notice config (managed here)

Table 15: Admin Panel — Files

9.3 Authentication

The Portal Manager implements a completely independent authentication flow from the student portal:

1. On every page load, the file checks `$_SESSION['is_admin']`. If not set or not `true`, it renders the admin login form and halts.
2. On login POST, the submitted username is looked up in the `admins` database table.

3. The submitted password is verified against the stored bcrypt hash using `password_verify()`.
4. On success, `$_SESSION['is_admin'] = true` and `$_SESSION['admin_id']` are set.
5. A `?logout=true` GET parameter destroys the session and redirects back to the login form.

> **Admin Credential Management:** Admin passwords are managed via `reset_admin_used.php` — a one-time utility that generates a bcrypt hash of a given plaintext password and upserts it into the `admins` table. This file must be deleted from the server after use as it exposes the new password in the response output.

9.4 Configuration Initialisation

On every authenticated page load, the Portal Manager checks for the existence of five configuration files and creates them with safe defaults if missing:

- `maintenance_config.json` → {`"maintenance_mode": "OFF", "reason": "Scheduled maintenance"`}
- `access_config.json` → {`"mode": "OPEN"`}
- `feedback_config.json` → Default schema with one text question
- `notice_config.json` → Default notice object with `status: "OFF"`

This self-initialisation ensures the portal cannot enter an undefined configuration state even if config files are accidentally deleted.

9.5 Flash Message System

All POST operations in the Portal Manager use a Post/Redirect/Get (PRG) pattern implemented via the `redirectWithFlash()` helper function:

1. The function stores the message and type (`success`, `warning`, or `error`) in `$_SESSION`.
2. It then redirects back to the same page URL (preserving any `path` or `chat_user` query parameters).
3. On the subsequent GET request, the session flash variables are read, displayed as a styled alert banner, and immediately unset.

This pattern prevents form resubmission on page refresh and keeps the URL clean.

9.6 Administrative Sections & Operations

9.6.1 Emergency Kill Switch (Maintenance Mode)

Purpose: Immediately blocks all student access to the portal with a custom maintenance message.

Working:

1. Admin enters a reason message and clicks "SHUT DOWN PORTAL."

2. The handler toggles `maintenance_mode` between `"ON"` and `"OFF"` in `maintenance_config.json` and updates the reason.
3. Every student-facing page (`index.php`, `forget_password.php`, `home.php`) reads `maintenance_config.json` on load. If `"ON"`, it immediately renders a full-screen maintenance notice and exits — no further code executes.
4. The maintenance notice displays the admin-provided reason and a "Check Again" button that reloads the page.
5. The Portal Manager itself is not affected — admin access is maintained during maintenance.

The header banner of the Portal Manager shows a live status indicator: **ONLINE** (green) or **OFFLINE** (red), reflecting the current maintenance state.

9.6.2 Global Popup Notice

Purpose: Push a one-time informational popup to all students on their next dashboard load.

Working:

1. Admin enters a title and message body (supports basic HTML in the message field).
2. A toggle switch enables or disables the notice.
3. On save, if the title or message has changed from the current config, a new `uniqid()` is generated — causing all students to see the notice again regardless of whether they dismissed a previous version.
4. If only the toggle changed, the existing ID is preserved — students who already dismissed the notice are not re-shown it.

9.6.3 Statistics Dashboard

Displays four real-time analytics widgets populated from the `log_entries` table:

- **Total Logs:** COUNT of all rows in `log_entries`.
- **Unique Students (Batch '25):** COUNT DISTINCT of Scholar Numbers starting with `'25%'`, filtering out test accounts and earlier batches.
- **Top 5 Resources:** GROUP BY `details` WHERE `event_type = 'Access'`, ordered by count DESC — reveals the most-viewed study material.
- **Recent Activity Log:** Last 10 log entries showing timestamp (timezone-corrected by +13h30m), Scholar Number, event type (colour-coded badge), and details.

9.6.4 Password Reset Requests

Purpose: Process student-submitted requests from `forget_password.php`.

Working:

1. Pending rows from `password_reset_requests` (WHERE `status = 'pending'`) are displayed in a table.
2. Each row shows: request timestamp, Scholar Number, request type (Forgot Password or Compromised Account), and a "View ID" link to the uploaded ID card image.
3. **Approve:** Resets the student's password to their Scholar Number (plain text), updates the request status to `'approved'`.
4. **Reject:** Updates the request status to `'rejected'` without modifying the student's password.
5. The sidebar shows a red badge with the count of pending requests.

9.6.5 Student Contributions (Upload Queue)

Covered in detail in Section 6. The sidebar badge shows the count of pending uploads.

9.6.6 Feedback System Management

Covered in detail in Section 7. Includes the Form Builder UI, response viewer, and data clearing.

9.6.7 Traffic Redirection

Purpose: Route students to different server instances based on their Roll Number group to distribute load across the hosting cluster.

Two-layer system:

Layer 1 — Database-based Server Cluster Control:

- A `redirection_settings` table stores a `mode` field and four target link fields (`target_link_2` through `target_link_5`).
- **Mode `off`:** All traffic stays on the primary server.
- **Mode `2_3`:** Sites 2 and 3 are active. Students from Sites 4 and 5 are redirected to the master.
- **Mode `4_5`:** Sites 4 and 5 are active. Students from Sites 2 and 3 are redirected to the master.
- This layer supports up to 5 server instances (primary + 4 mirrors).

Layer 2 — JSON-based Section URL Config:

- `redirection_config.json` maps student groups to URLs: `url_ae` for MT group (sections A–E) and `url_fj` for ST group (sections F–J).
- Toggling `status` to `"ON"` activates post-login redirection based on Roll Number section character (as described in Section 3).

Both layers are saved simultaneously on a single form submission in the Portal Manager.

9.6.8 Announcements Management

Covered in Section 8. Add/delete interface for `announcements.json`.

9.6.9 File Manager

Purpose: A web-based file manager for the `study_material/` directory, providing an alternative to FileZilla for quick administrative file operations without requiring FTP access.

Working:

- The current path is passed via `$_GET['path']`, defaulting to `study_material`.
- Directory traversal is prevented by comparing `realpath($currentPath)` against `realpath($BASE_DIR)` and checking for `'..'` in the path string.
- `scandir()` lists the current directory, separating results into folders and files.
- **Create Folder:** Folder names are sanitised with `preg_replace('/[^a-zA-Z0-9_-]/', '', ...)` before `mkdir()` is called.
- **Upload File:** Admin can upload a file directly into the current directory via a file input form.
- **Delete:** Both files and folders (recursively via `RecursiveDirectoryIterator`) can be deleted. All deletions are bounded by a security check that the target path resolves to within `BASE_DIR`.
- Folder icons are clickable links that navigate into the folder (`?path={currentPath}/{folder}#fi`
- An "Up Level" link navigates to the parent directory (`dirname($currentPath)`).

9.6.10 User Access Control

Provides two parallel user management interfaces:

Strict Access Mode Toggle: A large toggle switch controls the `access_config.json` mode between `"OPEN"` and `"STRICT"`. When enabled, a warning banner makes clear that *only* allowlisted users can log in. The Allowed Users panel is visually dimmed when STRICT mode is off, signalling it has no effect.

Allowed Users (Whitelist):

- Add: Scholar Number + note (name/label) is appended as a comma-separated line to `allowed_users.txt`.
- Remove: Lines containing the specified Scholar Number are filtered out of the file.

Banned Users:

- Ban: Scholar Number + reason is appended to `bannedusers.txt`.
- Unban: Lines containing the Scholar Number are filtered out.
- Both operations use the same file filter pattern: read all lines, exclude matching lines, overwrite file.

9.6.11 Student Messages

Purpose: A direct messaging interface between the admin and individual students, using a `messages` database table.

Working:

- A conversation list on the left shows the most recent message per student, ordered by recency. A red dot marks conversations where the last message was from the student (i.e. needs a reply).
- Clicking a conversation (or entering a Scholar Number in the search field) loads the full message thread for that student, displayed in a chat bubble layout (blue bubbles for admin, grey for student).
- The admin can type a reply and submit it. The reply is inserted into the `messages` table with `sender = 'admin'`.
- All timestamps in the messages view are adjusted by +13h30m for IST correction.

9.7 `maintenance_config.json` — System State

Field	Description
`maintenance_mode`	`"ON"` or `"OFF"` — controls portal-wide accessibility
`reason`	Human-readable explanation shown to students during maintenance

Table 16: `maintenance_config.json` Fields

10 Password Recovery & Account Assistance

The Login Assistance module provides a structured self-service flow for students who cannot access their accounts. It handles three distinct scenarios through a single page (`forget_password.php`): forgotten passwords, compromised accounts, and unregistered Scholar Numbers.

10.1 Overview

Unlike a typical automated password reset (email link, OTP), this system uses a **manual admin approval workflow** with ID card verification. This is a deliberate design choice given that MANIT does not provide institutional email addresses to all students, making automated email-based reset impractical. The trade-off is a processing delay (up to 24 hours) in exchange for verified identity confirmation.

10.2 `forget_password.php` — Account Assistance Page

10.2.1 Purpose

Serves a form with three selectable scenarios. Collects the student's Scholar Number and (for password scenarios) a college ID card image as proof of identity. Submits a request to the `password_reset_requests` database table for admin review.

10.2.2 Working — Maintenance Check

As the first executable statement, the file reads `maintenance_config.json`. If maintenance mode is `"ON"`, it renders the maintenance screen and exits — identical to the behaviour in `index.php`. This ensures the assistance page is also inaccessible during planned downtime.

10.2.3 Working — Three Scenarios

Case 1 — Forgot Password:

1. Student enters Scholar Number and uploads a college ID card (JPG/PNG/PDF, max 1MB).
2. Backend verifies the Scholar Number exists in the `students` table.
3. Checks for an existing pending request for this Scholar Number — if one exists, returns `"You already have a pending request"` to prevent duplicate submissions.
4. Validates the uploaded file: checks PHP upload error codes, enforces 1MB size limit, validates extension against `['jpg', 'jpeg', 'png', 'pdf']`.
5. Creates `uploads/id_cards/` directory if absent (with `0755` permissions).
6. Saves the file as `{scholar_no}_{time()}.{ext}` to prevent overwrites.
7. Inserts a row into `password_reset_requests` with type `"Forgot Password"`, timestamp, IP address, status `'pending'`, and the saved file path.
8. If DB insert fails, the uploaded file is deleted via `unlink()` to prevent orphaned files.

Case 2 — Compromised Account: Identical flow to Case 1 except the request type is `"Compromised Account"`. The UI uses red colour coding and the button reads "Report Compromise." The informational note states that account access will be frozen pending manual verification.

Case 3 — Scholar Number Not Found (New Registration): For students whose Scholar Number was not found in the `students` table (i.e. they exist at MANIT but were not seeded into the portal database):

1. Student enters only their Scholar Number (no file upload required).
2. Backend inserts a row into the `newregistration` table with the Scholar Number and timestamp.
3. Admin is notified and manually adds the student to the `students` table.
4. No duplicate check is performed on this path — if the Scholar Number is already pending, a duplicate entry error from MySQL prevents re-insertion (the table likely has a UNIQUE constraint on Scholar Number).

10.2.4 Working — UI Layer

The page renders a dropdown (`optionSelector`) with three options. JavaScript (`showPanel()`) hides all three form panels and reveals only the one matching the selected option. This creates a clean single-page experience without page reloads. File upload fields use a custom drag-and-drop styled area with JavaScript (`updateFileName()`) that hides the default file placeholder and shows a green checkmark with the selected filename once a file is chosen. A frontend size check (1MB) also fires at selection time to give immediate feedback before form submission.

10.3 Database Table — `password_reset_requests`

Column	Type	Description
`id`	INT (AUTO)	Primary key
`scholar_no`	VARCHAR	Requesting student's Scholar Number
`type`	VARCHAR	`'Forgot Password'` or `'Compromised Account'`
`request_time`	DATETIME	Timestamp of submission
`ip_address`	VARCHAR	Client IP at time of request
`status`	ENUM	`'pending'`, `'approved'`, `'rejected'`
`id_card_path`	VARCHAR	Server path to uploaded ID card file

Table 17: `password_reset_requests` Table Structure

11 Public Landing Page

11.1 `home.php` — Portal Landing Page

11.1.1 Purpose

A publicly accessible landing page that serves as the public face of the portal. Unlike `index.php` (the login page), `home.php` is designed for unauthenticated visitors and provides an overview of the portal, team information, and navigation to the login page. It uses the MANIT main building photo (`manit_main_building.jpg`) as a full-screen background.

11.1.2 Working

1. **Maintenance Check:** The first operation reads `maintenance_config.json`. If maintenance mode is `"ON"`, a plain HTML maintenance message is echoed and execution halts.
2. **Navigation Bar:** A fixed glassmorphism navbar with links to: About section, B.Tech New Scheme page, Syllabus PDF download, Team modal, and Student Login.
3. **Hero Section:** Full-viewport hero with the portal name and two CTA buttons — "Access Study Materials" (→ `index.php`) and "Check Announcements" (→ `#announcements` anchor).
4. **Announcements Card:** Dynamically populated from a hardcoded JavaScript array (academic resource PDFs). Uses a slide animation system with dot indicators. Note: this is separate from `announcements.json` — the landing page uses static JavaScript data.
5. **About Card:** A glassmorphism card describing the portal's purpose and providing quick links (Syllabus PDF, Student Login, Institute Website).
6. **Team Modal:** A click on "Team" in the nav triggers a modal overlay displaying the developer (Devansh Soni) and two core contributors (Tanay Sharma, Shree Pandit) with avatars, roles, and bios.
7. **Footer:** Standard footer with links and copyright year via `date("Y")`.

11.1.3 Design Notes

The landing page is entirely custom CSS (no Tailwind) using CSS variables and glassmorphism effects. It is the most visually elaborate page in the project. The background is a blurred overlay (`backdrop-filter: blur(8px)`) applied over the MANIT building photo. Navbar scroll behaviour is implemented in JavaScript — the background opacity increases on scroll.

12 Logging & Monitoring

The portal implements a pervasive event logging system that records all significant user actions to the `log_entries` database table. This provides the admin with a complete audit trail of portal usage and is the primary data source for the Statistics Dashboard in the Portal Manager.

12.1 Logged Events

Event Type	Logged By	Details Field
`Login`	`index.php`	Empty (IP + OS captured separately)
`Logout`	`logout.php`	Empty
`Access`	`resource_viewer.php`	`group/subject/filename`
`View Subjects`	`fetch_resources.php`	`"Viewed subjects in group: {group}"`
`View Folders`	`fetch_resources.php`	`"Viewed folders in subject: {subject}"`
`View Folder Content`	`fetch_resources.php`	`"Viewed folder: {folder} in subject: {subject}"`

Table 18: Portal Event Log Types

12.2 Database Table — `log_entries`

Column	Type	Description
`id`	INT (AUTO)	Primary key
`scholar_no`	VARCHAR	Acting student's Scholar Number
`event_type`	VARCHAR	Type of event (see above)
`event_time`	DATETIME	Server timestamp (requires +13h30m for IST)
`ip_address`	VARCHAR	Client IP address
`os`	VARCHAR	Detected client OS from User-Agent
`details`	VARCHAR	Event-specific detail string

Table 19: `log_entries` Table Structure

12.3 Client Fingerprinting

All logged events capture the client IP address and operating system. The IP detection priority chain (used consistently across `index.php`, `logout.php`, `fetch_resources.php`, and `resource_viewer.php`) is:

1. `HTTP_CLIENT_IP` — set by some proxies
2. `HTTP_X_FORWARDED_FOR` — standard reverse proxy header (first IP extracted)
3. `REMOTE_ADDR` — direct connection IP (fallback)

The OS detection uses a 15-entry regex array covering all major desktop and mobile platforms. The matched OS name is stored as a human-readable string (e.g. `"Windows 10"`, `"Android"`, `"iPhone"`).

13 Security Architecture

This section consolidates all security measures implemented across the portal, identifies known vulnerabilities, and provides recommendations for future hardening.

13.1 Implemented Security Measures

Measure	Implementation
Session fixation prevention	`session_regenerate_id(true)` called on every successful login
Persistent auth token	64-char cryptographically secure token via `bin2hex(random_bytes(32))`
HttpOnly cookie	`portal_auth` cookie set with `httponly=true, secure=true`
Session guards	Every protected PHP page checks `$_SESSION['scholarNo']` before any output
Prepared statements	All database queries use `$conn->prepare()` with bind_param to prevent SQL injection
Output escaping	All user-provided data rendered in HTML is escaped via `htmlspecialchars()`
Scholar No validation	Regex format check before any DB lookup in login handler
Directory traversal prevention	`basename()` sanitisation in `fetch_subjects.php`; `realpath()` comparison in `portal_manager.php` and `resource_viewer.php`
File type whitelist	Upload handler validates extension against an explicit allowlist
File size limits	10MB for student uploads, 1MB for ID card uploads
Admin password hashing	Admin credentials stored as bcrypt via `password_hash()`
Rate limiting	Student uploads limited to 10 files per day per Scholar Number
Client-side deterrents	Right-click, F12, Ctrl+Shift+I, Ctrl+U, Ctrl+S blocked via JavaScript

Table 20: Implemented Security Measures

13.2 Known Vulnerabilities & Gaps

Vulnerability	Location	Recommended Fix
Plain-text password storage	`students` table	Apply `password_hash()` on all student passwords; verify with `password_verify()`
No session token expiry enforcement	`session_tokens` table	Add a cron job or lazy expiry check that deletes tokens past their `expiry` date
Hardcoded DB credentials	`db_connection.php`	Move to environment variables or a config file outside webroot
Client-side security only	Dashboard, all pages	JS restrictions are trivially bypassable; rely on server-side controls for actual security
No CSRF protection	All POST forms	Add CSRF token to all forms; verify on POST
Hardcoded timezone offset	`attendance_handler.php`	Replace with `date_default_timezone_set('Asia/Kolkata`
No token expiry check on cookie auth	`index.php`	Query `expiry` field from `session_tokens` and reject expired tokens
`reset_admin_used.php` **left on server**	Root directory	Delete this file immediately after use

Table 21: Known Security Gaps & Recommendations

14 Known Issues & Future Roadmap

14.1 Known Issues

1. **Timezone drift:** The +13h30m hardcoded correction is empirically derived and will break if the hosting server changes its system clock or if the portal is migrated.
2. **Plain-text passwords:** All student passwords are stored as plain text in the database. A database compromise would expose all credentials.
3. **Session token expiry not enforced:** Tokens in `session_tokens` have an `expiry` column but it is never checked during cookie-based auto-login.
4. **Notice views file growth:** `notice_views.json` grows indefinitely. After each academic year, stale Scholar Numbers should be pruned.
5. **No CSRF protection:** All POST forms are vulnerable to cross-site request forgery.
6. **Filesystem-only study material:** There is no metadata layer (descriptions, tags, upload dates) for study material files — everything is inferred from folder

names.

7. **Redirection_status.txt is unused:** The file contains `"OFF"` and appears to be a legacy artifact not referenced by any current code.

14.2 Future Roadmap

1. **Password hashing migration:** Hash all existing plain-text passwords using bcrypt and update the comparison logic.
2. **CSRF token implementation:** Add a session-based CSRF token to all form submissions.
3. **Session token expiry enforcement:** Add expiry check in the cookie auto-login path.
4. **Proper IST timezone:** Replace the offset hack with `date_default_timezone_set('Asia/Kolk` if InfinityFree permits it, or with a UTC-based approach.
5. **Timetable integration for attendance:** Automatically detect which subjects have classes on which days to reduce manual entry burden.
6. **Mobile app:** A React Native or PWA wrapper around the portal for a native mobile experience.
7. **Email/WhatsApp notifications:** Automated alerts for new announcements, exam schedules, or PYQ additions.
8. **File metadata layer:** Add a database table for study material metadata (title, description, upload date, contributor) to improve searchability.
9. **Multi-batch support:** Extend Scholar Number validation patterns to accommodate future batches (2026, 2027) without code changes.
10. **Migrate to paid hosting:** InfinityFree's limitations (no persistent background jobs, shared MySQL, FTP-only management, traffic throttling) are fundamental constraints. Migration to a VPS or paid shared hosting would resolve the timezone issue, improve uptime, and enable background task support.

15 Configuration File Reference

All portal configurations are managed through flat files stored in the webroot. This table provides a complete reference.

File	Format	Purpose
`maintenance_config.json`	JSON Object	ON/OFF switch and reason for maintenance mode
`access_config.json`	JSON Object	OPEN/STRICT portal access mode
`redirection_config.json`	JSON Object	JSON-based group URL redirection (ON/OFF + URLs)
`redirection_status.txt`	Plain text	Legacy redirection flag (unused, value: OFF)
`announcements.json`	JSON Array	Dashboard announcement links
`notice_config.json`	JSON Object	Global popup notice content + unique ID
`notice_views.json`	JSON Object	Per-student notice dismissal tracking
`feedback_config.json`	JSON Object	Feedback form schema + ON/OFF status
`feedback_data.txt`	Plain text	Pipe-delimited feedback responses (append-only)
`feedback_submissions.json`	JSON Array	Scholar Numbers who submitted feedback
`bannedusers.txt`	CSV lines	Scholar No + reason for banned students
`allowed_users.txt`	CSV lines	Scholar No + note for whitelisted students

Table 22: Complete Configuration File Reference

16 Conclusion

The NITBFreshers Study Portal represents a comprehensive, independently developed academic resource platform built and maintained by students. From its initial deployment as a simple file-sharing site in 2024, it has evolved into a feature-rich system encompassing authentication, dynamic content delivery, attendance tracking, community contribution workflows, and a fully-featured administrative control panel.

The portal is architected around the constraints of its hosting environment — InfinityFree free shared hosting — and makes deliberate design choices to work within those constraints: a filesystem-backed content store requiring no database updates for new material, flat-file configuration for instant admin control without deployment cycles, and a multi-instance architecture for traffic distribution.

The codebase is entirely PHP procedural with no frameworks, making it accessible for maintenance by students with basic web development knowledge. The documentation in this document is intended to serve as a comprehensive handover reference for any future contributor, maintainer, or successor developer.

Portal URL: https://nitbfreshers.42web.io
Developer: Devansh Soni, B.Tech ECE (3rd Year), MANIT Bhopal
Documentation Version: 1.1 — April 2026
Total PHP Files: 20+ — **Total Config Files:** 12 — **Database Tables:** 9+